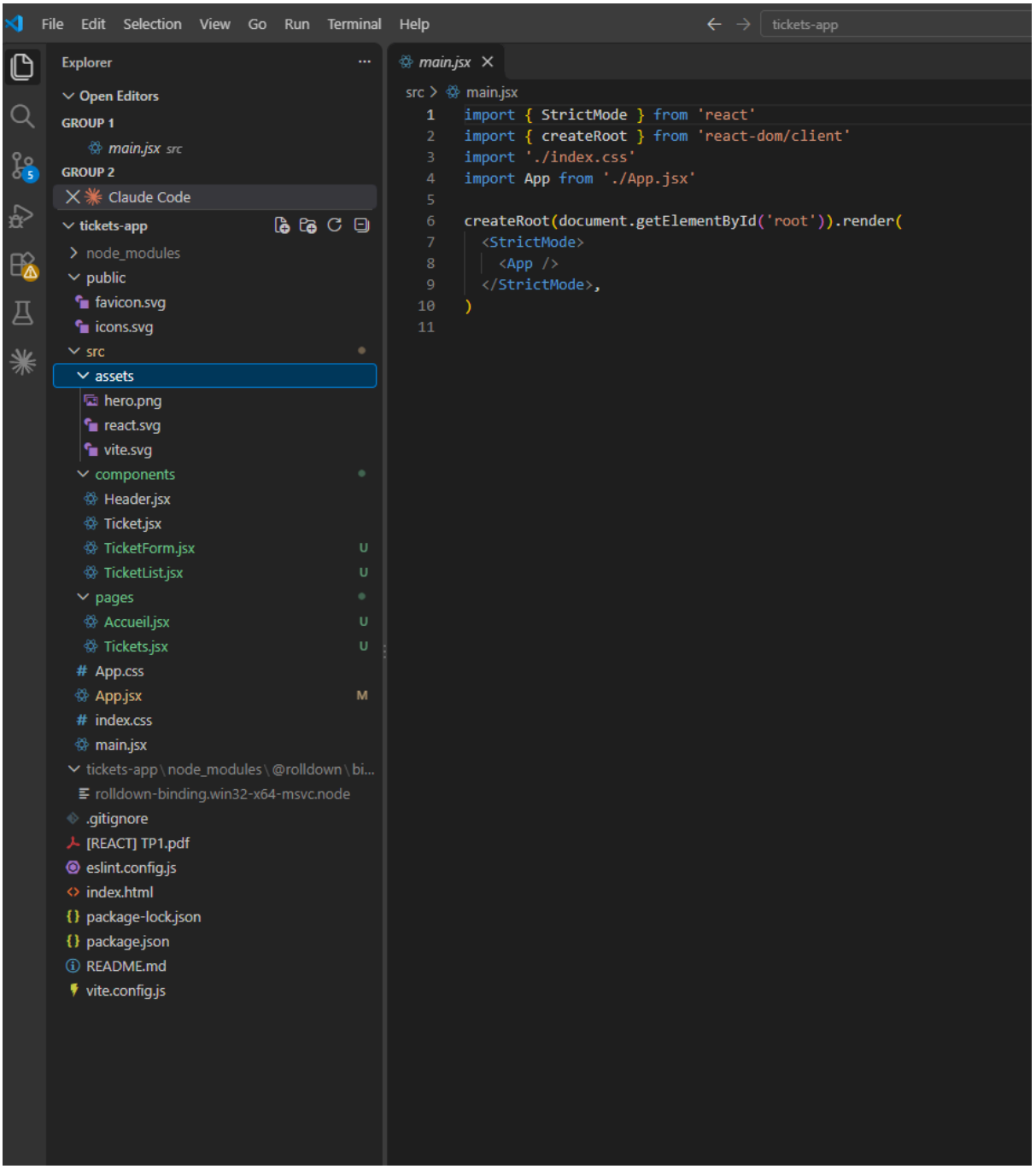
[REACT] TP2

Délio Duchesne-Brajon
Groupe 1

PARTIE 1 :

Question 1 : On stocke l'état dans le composant parent (App) car plusieurs composants ont besoin des mêmes données : TicketList les affiche et TicketForm doit pouvoir en ajouter. Comme les données circulent uniquement du parent vers l'enfant, deux composants frères ne peuvent pas partager un state : on le « remonte » donc dans leur parent commun, qui le transmet en props. On obtient ainsi une seule source de vérité, et TicketList devient un simple composant d'affichage.





Question 2 : Les données descendent par les props, les actions remontent par des fonctions de rappel (callbacks), et seul le propriétaire de l'état peut le modifier.

PARTIE 2 :

```
16    }
17
18    function changerStatut(id, nouveauStatut) {
19      setTickets(tickets.map((t) => (t.id === id ? { ...t, statut: nouveauStatut } : t)));
20    }
21
22    return (
23      <div>
24        <Header />
25        <Accueil />
26        <Tickets tickets={tickets} supprimerTicket={supprimerTicket} changerStatut={changerStatut} />
27      </div>
28    );
29  }
30
31  export default App;
32
```

src > pages > Tickets.jsx > ...

```
1   import TicketList from '../components/TicketList';
2
3   function Tickets({ tickets, supprimerTicket, changerStatut }) {
4     return (
5       <div>
6         <h2>Liste des tickets</h2>
7         <TicketList tickets={tickets} supprimerTicket={supprimerTicket} changerStatut={changerStatut}
8       </div>
9     );
10  }
```

```jsx
import Ticket from './Ticket';

function TicketList({ tickets, supprimerTicket, changerStatut }) {
  return (
    <div>
      {tickets.map((t) => (
        <Ticket key={t.id} {...t} supprimerTicket={supprimerTicket} changerStatut={changerStatut} />
      ))}
    </div>
```

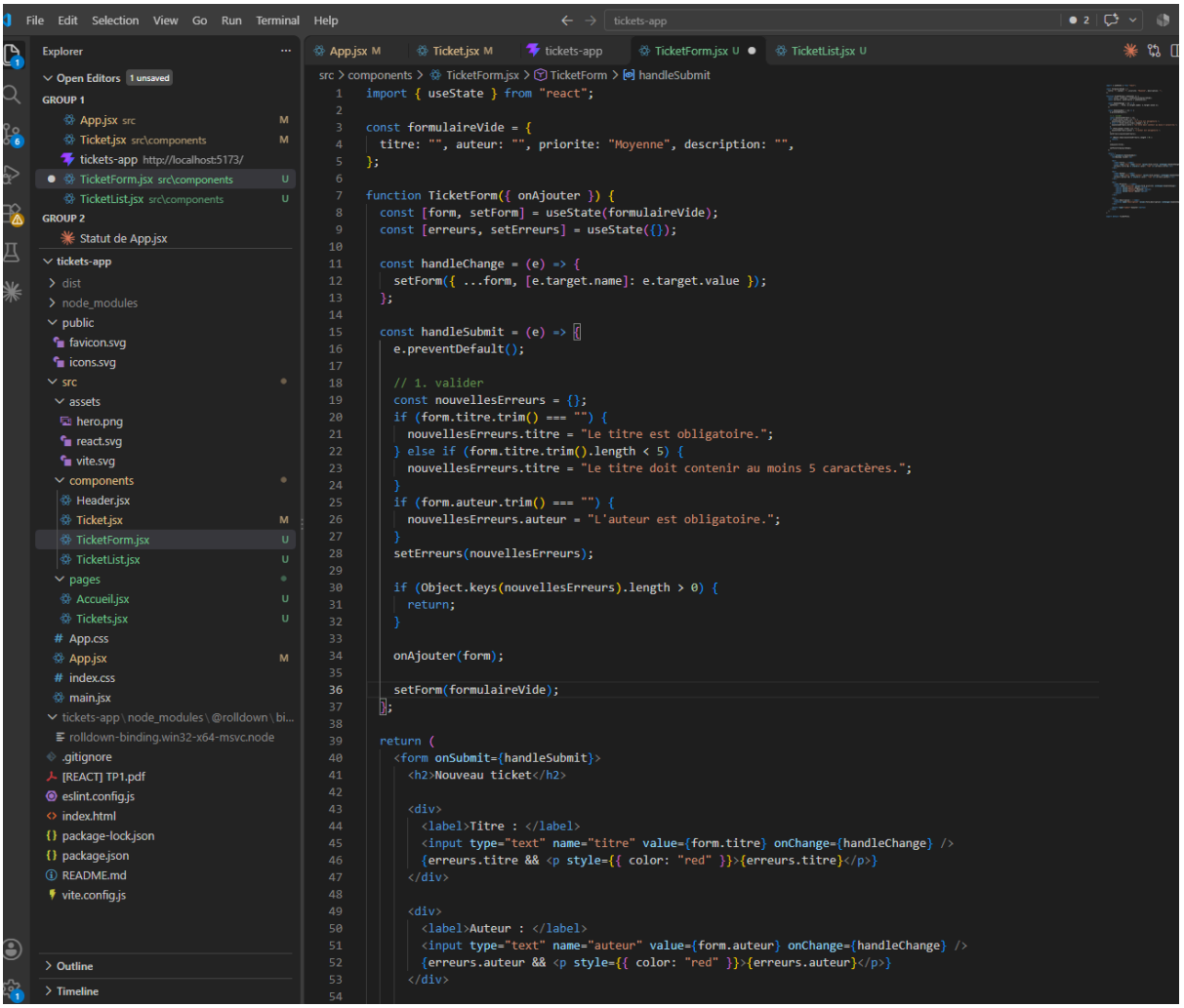
```jsx
function Ticket({ titre, auteur, priorite, statut, supprimerTicket, changerStatut, id }) {
  let couleur;
  if (statut === 'Ouvert') {
    couleur = 'red';
  } else if (statut === 'En cours') {
    couleur = 'orange';
  } else if (statut === 'Résolu') {
    couleur = 'green';
  }

  return (
    <div className="ticket">
      <h3>{titre}</h3>
      <p>Déclaré par : {auteur}</p>
      <p>Priorité : {priorite}</p>
      <p style={{ color: couleur }}>Statut : {statut}</p>
      <select value={statut} onChange={(e) => changerStatut(id, e.target.value)}>
        <option value="Ouvert">Ouvert</option>
        <option value="En cours">En cours</option>
        <option value="Résolu">Résolu</option>
      </select>
      <button onClick={() => supprimerTicket(id)}>Supprimer</button>
    </div>
  );
```

Question 3 : En React, on ne modifie jamais le state, on le remplace par une nouvelle copie. C'est cette nouvelle référence qui signale à React qu'il doit réafficher.

Question 4 : “...t” « étale » les propriétés de t dans un nouvel objet. Ça permet de faire une copie modifiée sans toucher à l'original.

PARTIE 3 :



Question 5 : Dans un composant contrôlé, React dit au champ quoi afficher ; dans un composant non contrôlé, c'est React qui demande au champ ce qu'il contient.

Question 6 : e.preventDefault() empêche le rechargement de la page à la soumission. Sans lui, le state de React serait effacé et le ticket ajouté disparaîtrait aussitôt.

Question 7 : [e.target.name]: e.target.value veut dire « mets à jour la propriété qui porte le nom du champ modifié, avec la valeur qui vient d'y être tapée ».

PARTIE 4 :

App.jsx M | Ticket.jsx M | tickets-app | TicketForm.jsx U | Tickets.jsx U | TicketList.jsx U

src > pages > Tickets.jsx > Tickets

```jsx
import { useState } from 'react';
import TicketList from '../components/TicketList';

const ordrePriorite = { Haute: 1, Moyenne: 2, Basse: 3 };

function Tickets({ tickets, supprimerTicket, changerStatut }) {
  const [recherche, setRecherche] = useState("");
  const [statutFiltre, setStatutFiltre] = useState("Tous");
  const [tri, setTri] = useState("aucun");

  const ticketsAffiches = tickets
    .filter((t) => t.titre.toLowerCase().includes(recherche.toLowerCase()))
    .filter((t) => statutFiltre === "Tous" || t.statut === statutFiltre);

  if (tri === "priorite") {
    ticketsAffiches.sort((a, b) => ordrePriorite[a.priorite] - ordrePriorite[b.priorite]);
  }

  return (
    <div>
      <h2>Liste des tickets</h2>
```

```jsx
      <label>Trier : </label>
      <select value={tri} onChange={(e) => setTri(e.target.value)}>
        <option value="aucun">Aucun tri</option>
        <option value="priorite">Priorité (Haute &gt; Moyenne &gt; Basse)</option>
      </select>
    </div>

    <p>{ticketsAffiches.length} ticket(s) affiché(s) sur {tickets.length}</p>

    <TicketList tickets={ticketsAffiches} supprimerTicket={supprimerTicket} changerStatut={changerStatut
  </div>
);
}

export default Tickets;
```

Nouveau ticket

Titre :
Auteur :
Priorité : Moyenne
Description :
Ajouter

Liste des tickets

Rechercher : vpn
Statut : Tous
Trier : Aucun tri

1 ticket(s) affiché(s) sur 3

Accès VPN refusé

Déclaré par : Mme Martin

Priorité : Moyenne

Statut : En cours

En cours Supprimer

Rechercher : Titre du ticket
Statut : Tous
Trier : Priorité (Haute > Moyenne > Basse)

2 ticket(s) affiché(s) sur 2

Imprimante en panne

Déclaré par : M. Durand

Priorité : Haute

Statut : Ouvert

Ouvert Supprimer

Écran qui clignote

Déclaré par : M. Petit

Priorité : Basse

Statut : Résolu

Résolu Supprimer

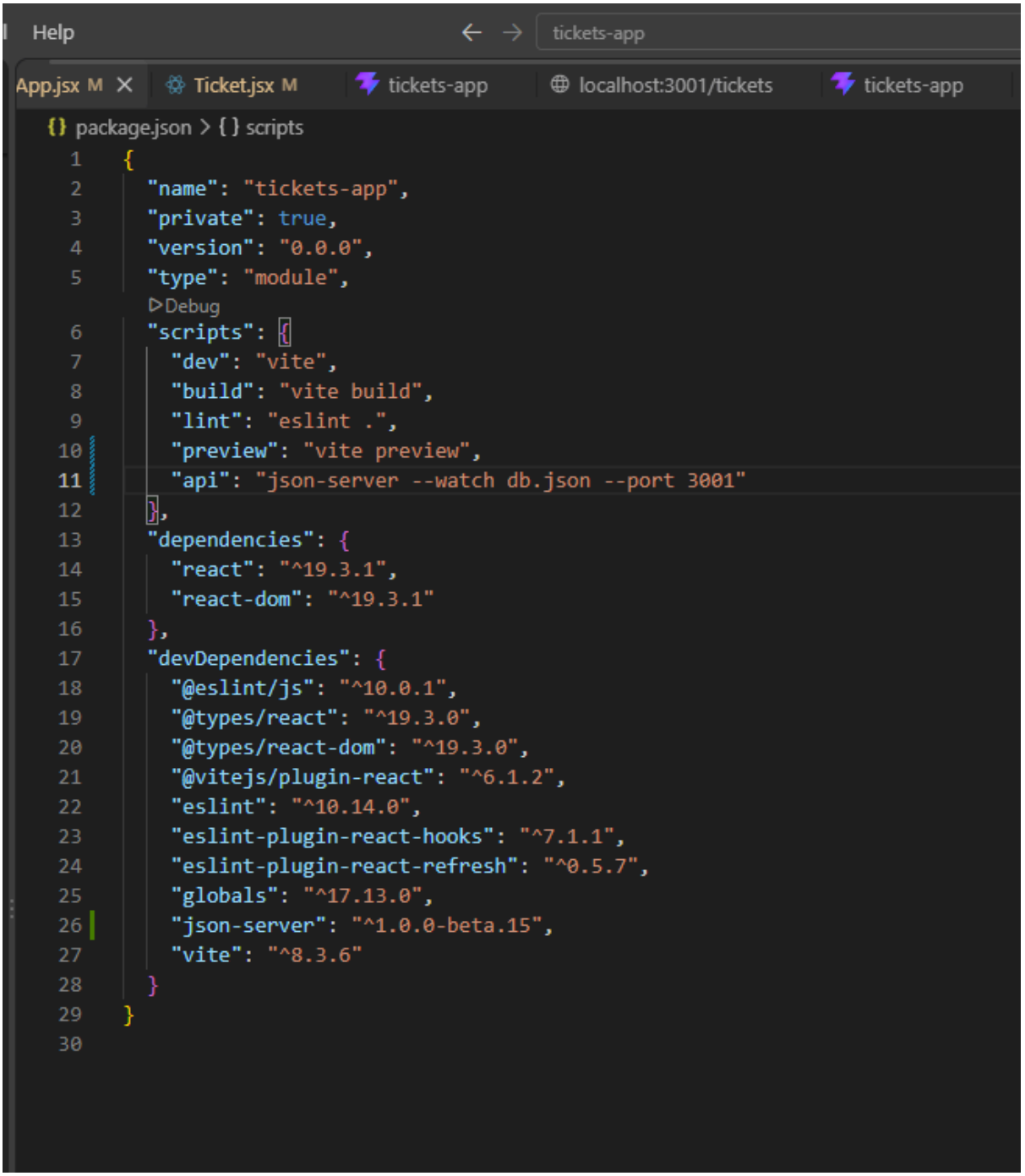
Question 8 : Stocker la liste filtrée, c'est dupliquer des données qu'il faudra garder synchronisées à la main ; la calculer garantit qu'elle est toujours à jour, avec moins de code.

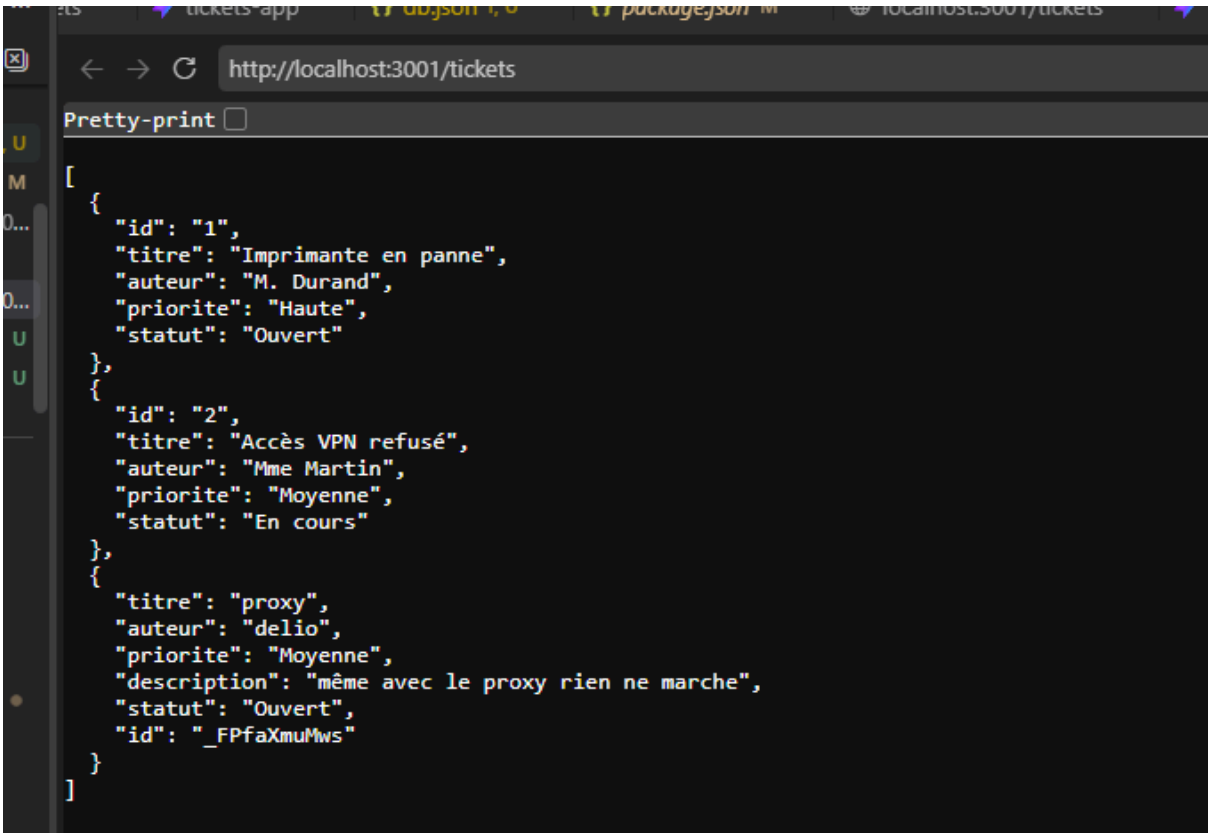
PARTIE 5 :

App.jsx M | Ticket.jsx M | tickets-app | localhost:3001/tickets | tickets-app | db.json U | TicketForm.jsx U

db.json > ...

```
{
  "tickets": [
    { "id": 1, "titre": "Imprimante en panne", "auteur": "M. Durand", "priorite": "Haute", "statut": "Ouve
    { "id": 2, "titre": "Accès VPN refusé", "auteur": "Mme Martin", "priorite": "Moyenne", "statut": "En c
  ]
}
```





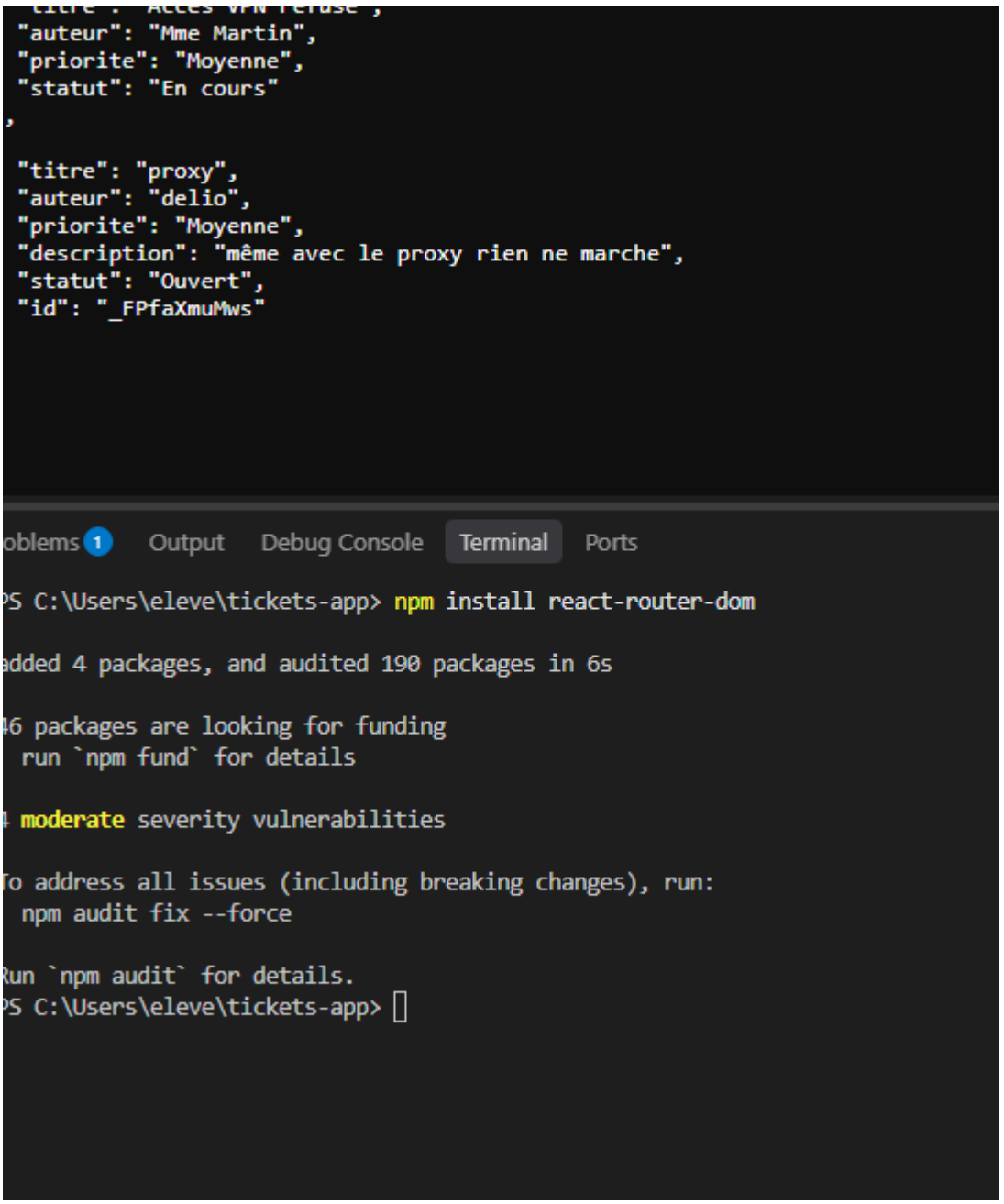
Question 9 : le tableau de dépendances dit à React « relance l'effet seulement si ces valeurs changent » ; `[]` veut dire « jamais après la première fois », et sans tableau l'effet est relancé à chaque affichage, ce qui crée ici une boucle infinie de requêtes.

Question 10 :

Verbe HTTP	CRUD	Rôle
POST	**C**reate (créer)	Ajouter une nouvelle ressource
GET	**R**ead (lire)	Récupérer une ou plusieurs ressources
PUT	**U**pdate (modifier)	Remplacer **toute** la ressource
DELETE	**D**elete (supprimer)	Supprimer une ressource

Question 11 : `useEffect` attend une fonction qui renvoie rien ou une fonction de nettoyage ; une fonction `async` renvoie toujours une Promise, donc on déclare la fonction `async` à l'intérieur de l'effet et on l'appelle.

PARTIE 6 :


"titre": "Acces VPN refuse",
"auteur": "Mme Martin",
"priorite": "Moyenne",
"statut": "En cours"
},
"titre": "proxy",
"auteur": "delio",
"priorite": "Moyenne",
"description": "même avec le proxy rien ne marche",
"statut": "Ouvert",
"id": "_FPfaXmuMws"
oblems 1
Output
Debug Console
Terminal
Ports
PS C:\Users\eleve\tickets-app> npm install react-router-dom
added 4 packages, and audited 190 packages in 6s
46 packages are looking for funding
run `npm fund` for details
4 moderate severity vulnerabilities
To address all issues (including breaking changes), run:
npm audit fix --force
Run `npm audit` for details.
PS C:\Users\eleve\tickets-app>

Help tickets-app

tickets-app | db.json 1, U | localhost:3001/tickets | tickets-app

src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

Help tickets-app

App.jsx M | Ticket.jsx M | tickets-app | localhost:3001/tickets | tickets-app | db.json 1, U | localhost:3001/

src > App.jsx > App

```jsx
function App() {
  async function ajouterTicket(form) {
    if (!reponse.ok) throw new Error("Erreur " + reponse.status);
    const nouveauTicket = await reponse.json();
    setTickets((anciens) => [...anciens, nouveauTicket]);
    navigate("/tickets");
  } catch (e) {
    setErreur(e.message);
  }
}

return (
  <div>
    <Header />
    {chargement && <p>Chargement...</p>}
    {erreur && <p style={{ color: "red" }}>Problème avec l'API : {erreur}</p>}
    {!chargement && !erreur && (
      <Routes>
        <Route path="/" element={<Accueil />} />
        <Route
          path="/tickets"
          element={<Tickets tickets={tickets} supprimerTicket={supprimerTicket} changerStatut={changerSt
        />
        <Route path="/tickets/nouveau" element={<TicketForm onAjouter={ajouterTicket} />} />
        <Route path="/tickets/:id" element={<DetailTicket tickets={tickets} />} />
        <Route path="*" element={<PageIntrouvable />} />
      </Routes>
    )}
  </div>
);
}

export default App;
```

App.jsx M | Header.jsx M | Ticket.jsx M | tickets-app | localhost:3001/tickets

src C:\Users\eleve\tickets-app\src\App.jsx • Modified

```jsx
import { NavLink } from 'react-router-dom';

function Header() {
  return (
    <header>
      <h1>Gestion des tickets</h1>
      <nav>
        <NavLink to="/">Accueil</NavLink>
        {" | "}
        <NavLink to="/tickets" end>Tickets</NavLink>
        {" | "}
        <NavLink to="/tickets/nouveau">Nouveau ticket</NavLink>
      </nav>
    </header>
  );
}

export default Header;
```

App.jsx M | DetailTicket.jsx U | Ticket.jsx M | tickets-app

src > pages > DetailTicket.jsx > ...

```jsx
import { useParams } from 'react-router-dom';

function DetailTicket({ tickets }) {
  const { id } = useParams();
  const ticket = tickets.find((t) => t.id === id);

  if (!ticket) {
    return <p>Ticket introuvable.</p>;
  }

  return (
    <div>
      <h2>{ticket.titre}</h2>
      <p>Déclaré par : {ticket.auteur}</p>
      <p>Priorité : {ticket.priorite}</p>
      <p>Statut : {ticket.statut}</p>
      <p>Description : {ticket.description}</p>
    </div>
  );
}

export default DetailTicket;
```

src > App.jsx > App > ajouterTicket

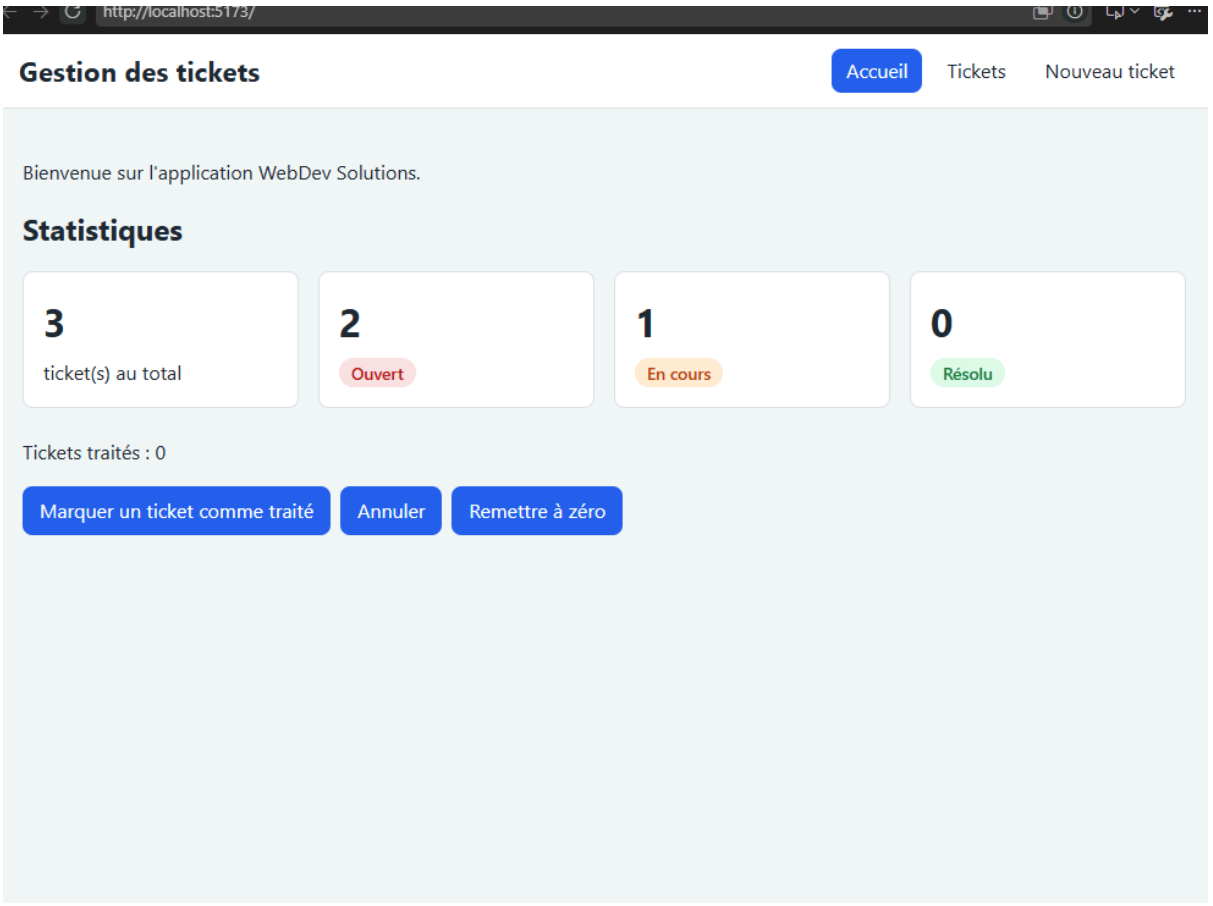
```jsx
function App() {
  async function changerStatut(id, nouveauStatut) {
      const reponse = await fetch(`${API}/${id}`, {
        body: JSON.stringify({ statut: nouveauStatut }),
      });
      if (!reponse.ok) throw new Error("Erreur " + reponse.status);
      setTickets((anciens) => anciens.map((t) => (t.id === id ? { ...t, statut: nouveauStatut } : t)));
    } catch (e) {
      setErreur(e.message);
    }
  }

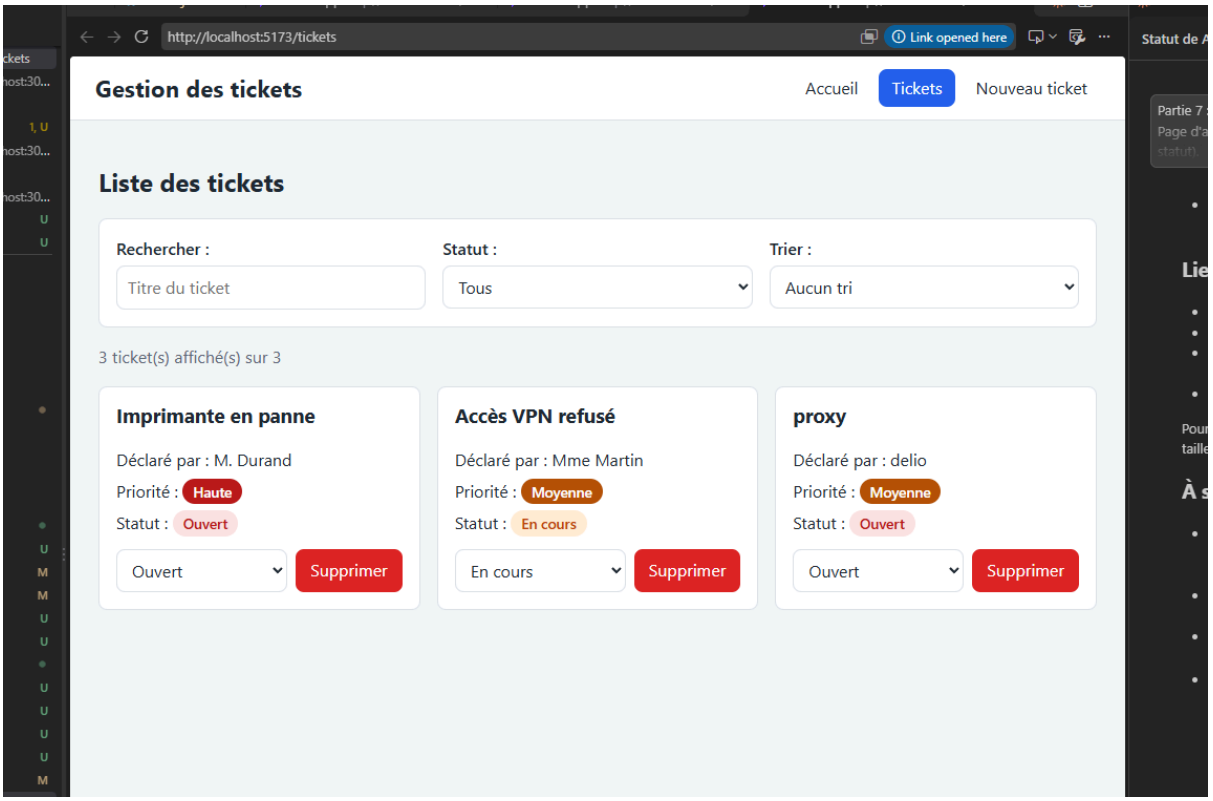
  async function ajouterTicket(form) {
    try {
      const reponse = await fetch(API, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ ...form, statut: "Ouvert" }),
      });
      if (!reponse.ok) throw new Error("Erreur " + reponse.status);
      const nouveauTicket = await reponse.json();
      setTickets((anciens) => [...anciens, nouveauTicket]);
      navigate("/tickets");
    } catch (e) {
      setErreur(e.message);
    }
  }

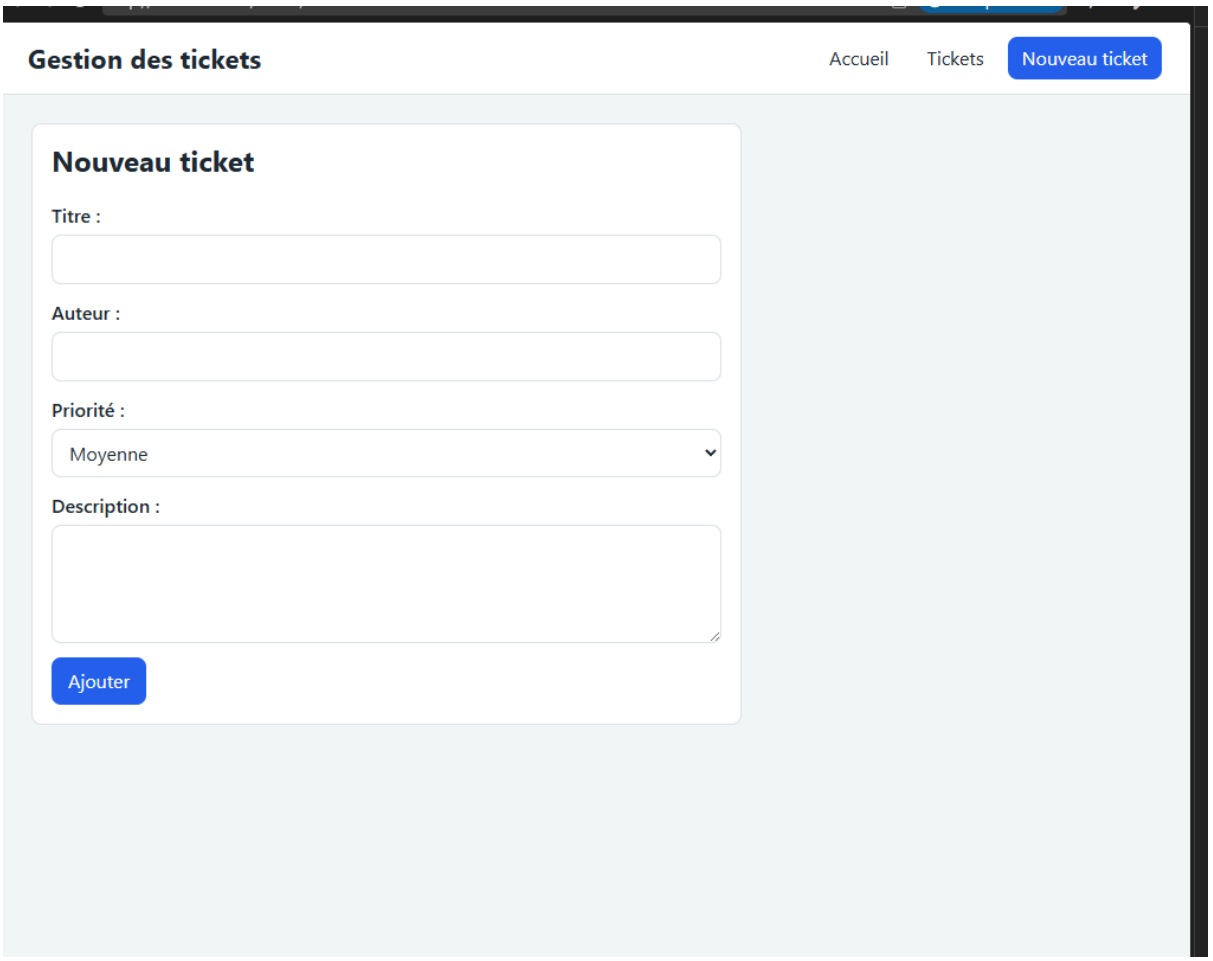
  return (
```

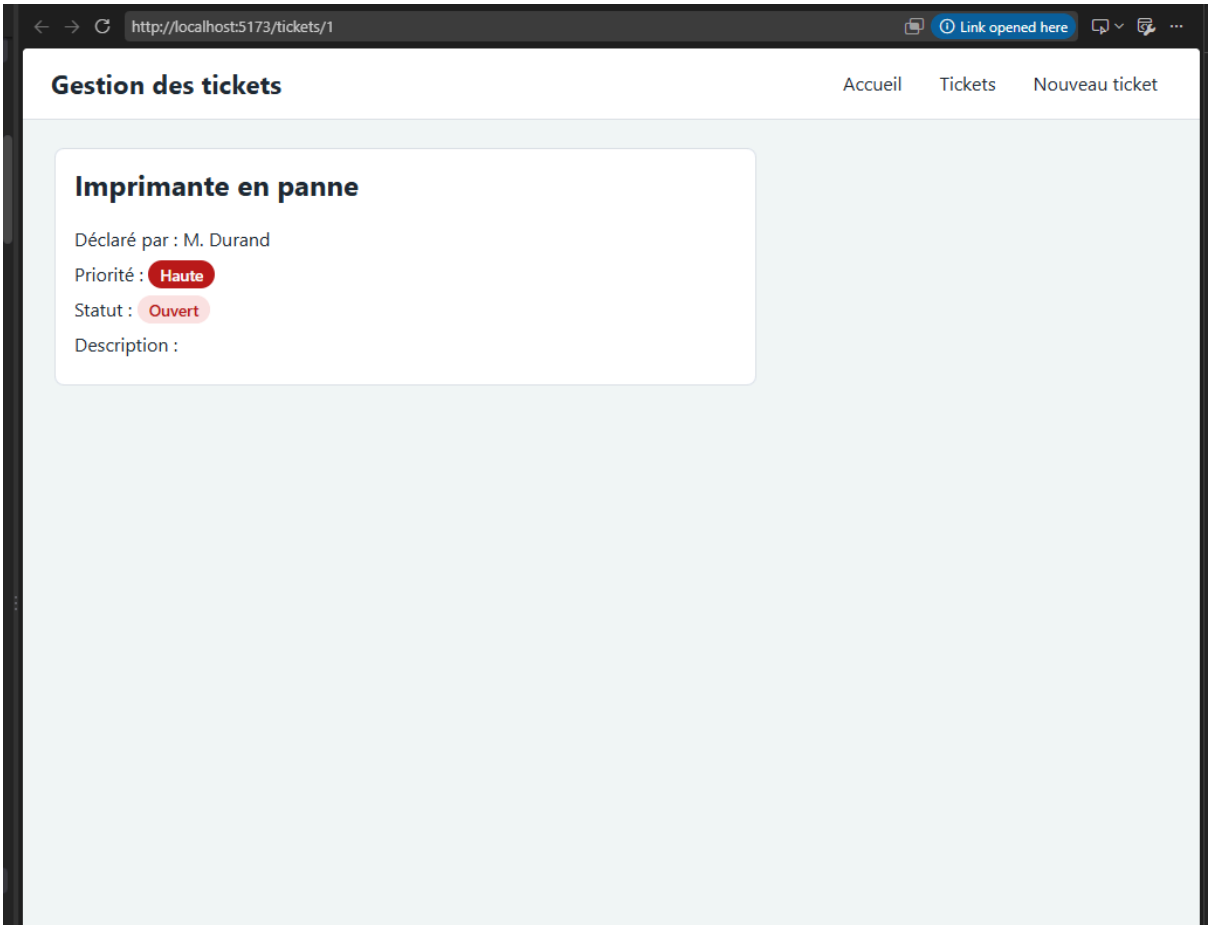
Question 12 : Quelle différence entre `<a href>` et `<Link>` ? Quel est l'intérêt pour une SPA (*Single Page Application*) ?

PARTIE 7 :


http://localhost:5173/
Gestion des tickets
Accueil
Tickets
Nouveau ticket
Bienvenue sur l'application WebDev Solutions.
Statistiques
3
ticket(s) au total
2
Ouvert
1
En cours
0
Résolu
Tickets traités : 0
Marquer un ticket comme traité
Annuler
Remettre à zéro


http://localhost:5173/tickets
Link opened here
Gestion des tickets
Accueil
Tickets
Nouveau ticket
Liste des tickets
Rechercher :
Titre du ticket
Statut :
Tous
Trier :
Aucun tri
3 ticket(s) affiché(s) sur 3
Imprimante en panne
Déclaré par : M. Durand
Priorité : Haute
Statut : Ouvert
Ouvert
Supprimer
Accès VPN refusé
Déclaré par : Mme Martin
Priorité : Moyenne
Statut : En cours
En cours
Supprimer
proxy
Déclaré par : delio
Priorité : Moyenne
Statut : Ouvert
Ouvert
Supprimer


Gestion des tickets
Accueil
Tickets
Nouveau ticket
Nouveau ticket
Titre :
Auteur :
Priorité :
Moyenne
Description :
Ajouter


http://localhost:5173/tickets/1
Link opened here
Gestion des tickets
Accueil
Tickets
Nouveau ticket
Imprimante en panne
Déclaré par : M. Durand
Priorité : Haute
Statut : Ouvert
Description :